



Приложение Б. Инструментарий

Введение

Профессиональная разработка — это не только написание кода, но и **управление сложностью** вокруг него. Как вы убедились в Модулях 4–11, архитектура FastAPI-приложения может быть безупречной, но если:

- форматирование кода разнится от файла к файлу,
- зависимости конфликтуют при каждом `pip install`,
- типовые ошибки обнаруживаются только в продакшене,
- команды для тестов, миграций и деплоя живут в голове одного разработчика,

— то скорость команды падает, а технический долг растёт экспоненциально.

Это приложение — про **toolchain**: инструменты, которые превращают написание кода из ремесла в инженерную дисциплину. Мы разберём не только «как пользоваться», но и **почему** эти инструменты работают так, а не иначе: от AST-деревьев в линтерах до NP-полноты разрешения зависимостей.

Б.1. Качество кода: линтеры, форматтеры, типизация, pre-commit

Б.1.1. `ruff`: линтер как конечный автомат над AST

Что такое линтер? Линтер — это программа, которая анализирует исходный код без его выполнения (static analysis) и находит потенциальные ошибки, нарушения стиля или антипаттерны.

Как работает линтер под капотом?

1. **Лексический анализ (tokenization)**: исходный текст разбивается на токены (`def` , `class` , `x` , `=` , `42`).
2. **Синтаксический анализ (parsing)**: токены превращаются в **AST**

(Abstract Syntax Tree, абстрактное синтаксическое дерево).

3. **Семантический анализ:** обход AST с проверкой правил.

AST — это дерево, где внутренние узлы — операторы (`FunctionDef` , `BinOp`), а листья — операнды (`Name` , `Constant`). Например, код `x = 1 + 2` превращается в:

```
In [ ]: Assign
├─ targets: [Name(id='x')]
└─ value: BinOp
    ├─ left: Constant(value=1)
    ├─ op: Add()
    └─ right: Constant(value=2)
```

Линтер обходит это дерево (обычно DFS, $O(n)$, где n — число узлов) и проверяет правила: «нет ли неиспользуемых переменных?», «нет ли импортов вне top-level?».

Почему `ruff` ?

До `ruff` Python-экосистема использовала комбинацию:

- `flake8` — линтер (pyflakes + pycodestyle + mccabe)
- `pylint` — расширенный линтер с семантическим анализом
- `isort` — сортировка импортов
- `autoper8` / `yapf` — форматирование
- `bandit` — поиск уязвимостей

Проблема: каждый инструмент парсит AST **самостоятельно**. На кодовой базе в 100 000 строк запуск всех инструментов занимает 10–30 секунд.

`ruff` (написан на Rust) объединяет 500+ правил из `flake8` , `pylint` , `isort` , `pyupgrade` , `bandit` и работает в **едином проходе** по AST. Сложность остаётся $O(n)$, но константа уменьшена на порядок.

```
In [ ]: # pyproject.toml
[tool.ruff]
target-version = "py312" # Python 3.12
line-length = 88

[tool.ruff.lint]
select = [
    "E", # pycodestyle errors
    "W", # pycodestyle warnings
    "F", # Pyflakes
    "I", # isort
```

```

"N", # pep8-naming
"W", # pycodestyle warnings
"UP", # pyupgrade
"B", # flake8-bugbear
"C4", # flake8-comprehensions
"SIM", # flake8-simplify
]
ignore = ["E501"] # line too long — пусть black решает

[tool.ruff.lint.pydocstyle]
convention = "google"

[tool.ruff.format]
quote-style = "double"
indent-style = "space"

```

Ключевая идея: `ruff` не просто быстрее — он **единый**. Одно AST-дерево, один проход, один конфиг. Это уменьшает энтропию toolchain'a.

Б.1.2. `black`: каноническая форма кода

Проблема: споры о форматировании («таб vs пробелы», «где переносить скобку») отнимают время на код-ревью и вносят шум в diff'ы.

Решение: `black` — opinionated formatter. Он не спрашивает. Он берёт ваш код и выдаёт **единственно возможную** каноническую форму. Это как приведение матрицы к ступенчатому виду: результат детерминирован.

```

In [ ]: # До black
def my_function(x,y):
    if x>0:
        return x+y
    else:
        return x-y

# После black
def my_function(x, y):
    if x > 0:
        return x + y
    else:
        return x - y

```

Почему line-length = 88? Гвидо ван Россум в PEP 8 рекомендовал 79 символов (ширина терминала 1980-х). `black` выбрал 88, потому что это оптимум: на 10% больше 80, что снижает число переносов на ~20%, при этом строка всё ещё помещается в современные редакторы.

Математическая подоплёка: каноническая форма

Пусть C — множество всех синтаксически корректных программ на Python. Форматирование — это отношение эквивалентности: две программы эквивалентны, если имеют одинаковый AST. `black` определяет **функцию канонизации**:

$\text{black}: C \rightarrow C_{\text{canonical}}$

где $C_{\text{canonical}} \subset C$ — множество программ в «чёрном» стиле.

Свойство: $\forall p \in C: \text{black}(\text{black}(p)) = \text{black}(p)$ —

идемпотентность. Это критично: повторный запуск `black` не меняет результат.

Интеграция:

```
In [ ]: # Форматировать всё
        black app/ tests/

        # Проверить, что всё отформатировано (в CI)
        black --check app/ tests/
```

Б.1.3. `mypy`: статическая типизация и soundness

Python — динамически типизированный язык. Но аннотации типов (PEP 484) позволяют **статическому анализатору** проверять корректность до запуска.

Теория типов: sound vs unsound

- **Sound type system:** если программа проходит типизацию, она не выдаст type error во время выполнения. Пример: Haskell, Rust.
- **Unsound type system:** статический анализатор может пропустить ошибку, которая произойдёт в runtime. Пример: `mypy` с Python.

Python не может быть sound, потому что:

- `getattr(obj, 'nonexistent')` — валиден синтаксически, но может упасть.
- `eval()`, `exec()` — произвольный код.
- С-расширения могут нарушать инварианты.

Но `mypy` даёт **gradual typing** (постепенную типизацию): вы добавляете типы туда, где это важно, и получаете защиту. Это компромисс между

гибкостью Python и безопасностью статически типизированных языков.

```
In [ ]: # мупу поймает это до запуска
def greet(name: str) -> str:
    return f"Hello, {name}"

greet(42) # мупу: Argument 1 to "greet" has incompatible type "int"; exp
```

Конфигурация:

```
In [ ]: # pyproject.toml
[tool.mypy]
python_version = "3.12"
strict = true
warn_return_any = true
warn_unused_ignores = true
ignore_missing_imports = true # для библиотек без stubs

plugins = [
    "pydantic.mypy",
    "sqlalchemy.ext.mypy.plugin",
]
```

Строгий режим (`strict`): включает:

- `--disallow-untyped-defs` — все функции должны иметь аннотации.
- `--disallow-any-generics` — `list` вместо `list[Any]`.
- `--warn-redundant-casts` — лишние `cast()`.

Почему это важно для FastAPI: FastAPI и Pydantic построены на типах.

Если `мупу` не проходит — скорее всего, Pydantic тоже сломается на границе системы.

Б.1.4. `pre-commit`: fail-fast на клиенте

Git hooks — скрипты, запускаемые Git при определённых событиях.

`pre-commit` хук запускается **до создания коммита**. Если хук падает с ошибкой — коммит не создаётся.

Почему это критично: исправить ошибку на вашей машине стоит 10 секунд. Исправить её после падения CI — 10 минут (контекст-переключение, ожидание, новый коммит, новый pipeline).

```
In [ ]: # .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
```

```

rev: v0.6.0
hooks:
  - id: ruff
    args: [--fix]
  - id: ruff-format

- repo: https://github.com/pre-commit/mirrors-mypy
  rev: v1.11.0
  hooks:
    - id: mypy
      additional_dependencies: [pydantic, sqlalchemy, types-redis]

- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-added-large-files
      args: [--maxkb=1000]

```

Как это работает:

```

In [ ]: # Установка хуков (один раз на репозиторий)
pre-commit install

# Ручной запуск на всех файлах
pre-commit run --all-files

# При git commit:
# 1. ruff проверяет staged файлы
# 2. mypy проверяет staged файлы
# 3. Если всё ОК — коммит создаётся
# 4. Если нет — коммит отменяется, вы видите ошибки

```

Математическая подоплёка: `pre-commit` реализует **инвариант** на множестве коммитов: $\forall c \in \{\text{Commits}\}: \text{ruff}(c) \wedge \text{mypy}(c)$. Это гарантирует, что история Git всегда проходит базовые проверки.

Б.2. Управление зависимостями: `poetry`, `pdm`, `uv`

Б.2.1. Проблема: dependency hell и SAT

Транзитивные зависимости: если ваш проект зависит от `A==1.0`, а `A` зависит от `B>=2.0,<3.0`, а `B` зависит от `C>=1.0` — в вашем окружении

оказываются `A`, `B`, `C`, даже если вы не просили `C` напрямую.

Конфликт версий: если `A` требует `B>=2.0`, а `D` требует `B<2.0` — **невозможно** удовлетворить оба требования. Это называется **dependency hell**.

Математическая подоплёка: SAT-проблема

Разрешение зависимостей — это частный случай **SAT** (Boolean Satisfiability) или, точнее, **CSP** (Constraint Satisfaction Problem). Для каждого пакета p_i множество допустимых версий $V_i \subseteq \mathbb{N}$. Для каждой зависимости $p_i \rightarrow p_j$ задано ограничение $v_j \in R_{ij}(v_i)$. Нужно найти:

$$\exists (v_1, \dots, v_n) \in V_1 \times \dots \times V_n: \forall (i,j) \in E: v_j \in R_{ij}(v_i)$$

SAT — NP-полная задача. На практике менеджеры зависимостей используют эвристики (backtracking, unit propagation), а не полный перебор. Но в Python-экосистеме с тысячами пакетов разрешение может занимать минуты.

Lock-файл: решение проблемы — зафиксировать **точный** набор версий, который однажды разрешён. Это делает сборку **детерминированной**: `lock-file` `environment` однозначно.

Б.2.2. poetry: стандарт де-факто

`poetry` использует `pyproject.toml` (PEP 518, PEP 621) как единый источник правды: зависимости, метаданные, скрипты, конфигурация инструментов.

```
In [ ]: # pyproject.toml
[tool.poetry]
name = "ml-api"
version = "0.1.0"
description = "Async ML backend"
authors = ["Team <team@example.com>"]

[tool.poetry.dependencies]
python = "^3.12"
fastapi = "^0.111.0"
pydantic = "^2.8.0"
sqlalchemy = {extras = ["asyncpg"], version = "^2.0.0"}

[tool.poetry.group.dev.dependencies]
```

```

pytest = "^8.0.0"
pytest-asyncio = "^0.23.0"
mypy = "^1.11.0"
ruff = "^0.6.0"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"

```

Команды:

```

In [ ]: # Установка зависимостей из lock-файла
        poetry install

        # Добавление зависимости
        poetry add httpx

        # Добавление dev-зависимости
        poetry add --group dev pytest-mock

        # Обновление lock-файла
        poetry lock

        # Запуск внутри виртуального окружения
        poetry run uvicorn app.main:app

        # Активация shell
        poetry shell

```

Как poetry разрешает зависимости: использует алгоритм, похожий на **PubGrub** (изначально из Dart), с подробными сообщениями о конфликтах.

Б.2.3. pdm : PEP 582 и локальные пакеты

pdm — альтернатива poetry, ориентированная на стандарты PEP:

- PEP 582: `__pypackages__` — локальная директория с пакетами, без виртуального окружения (экспериментально).
- Нативная поддержка editable installs.
- Лучшая интеграция с `pdm-backend` для сборки пакетов.

```

In [ ]: pdm init          # интерактивное создание pyproject.toml
        pdm add fastapi
        pdm install
        pdm run python -m pytest

```

Когда pdm лучше poetry: если вы разрабатываете библиотеку (а не

приложение) и нужна максимальная совместимость со стандартами Python Packaging Authority.

Б.2.4. uv : революция скорости

`uv` — инструмент от Astral (создатели `ruff`), написанный на Rust. Он заменяет:

- `pip` (установка пакетов)
- `venv` / `virtualenv` (создание окружений)
- `poetry` / `pdm` (разрешение и lock)

Скорость: `uv` устанавливает зависимости в **10-100 раз** быстрее `pip`. На холодном кэше установка 100 пакетов занимает секунды вместо минут.

Почему так быстро?

1. Параллельная загрузка и установка (Rust async).
2. Глобальный кэш на уровне пользователя (не дублирует пакеты между проектами).
3. Оптимизированный разбор wheel'ов (zip-файлов).

Команды:

```
In [ ]: # Создание окружения и установка
uv venv
uv pip install -r requirements.txt

# Или через pyproject.toml (как poetry)
uv sync                # установка из lock-файла
uv lock                 # создание uv.lock
uv add fastapi          # добавление зависимости
uv run uvicorn app.main:app # запуск внутри окружения
```

uv vs poetry : `uv` быстрее, но моложе (2024). `poetry` зрелее, больше плагинов. Для новых проектов `uv` — рекомендация.

Математическая подоплёка: PubGrub

`uv` использует алгоритм **PubGrub** для разрешения зависимостей. Это алгоритм на основе **conflict-driven clause learning (CDCL)** из SAT-решателей:

1. Выбирается пакет с наименьшим числом версий.
2. Пробуется версия. Если конфликт — запоминается **причина** конфликта (clause).
3. Причина используется для отсечения целых поддеревьев поиска (backjumping вместо naive backtracking).

Сложность в худшем случае остаётся экспоненциальной, но на практике PubGrub на порядок быстрее простого backtracking.

Б.3. Автоматизация команд: `just`, `make`

Б.3.1. `make`: классика с ограничениями

`make` — инструмент сборки из 1976 года. Он использует **Makefile** с правилами:

```
In [ ]: # Makefile
.PHONY: test lint format migrate

test:
    pytest -xvs tests/

lint:
    ruff check app/ tests/
    mypy app/

format:
    ruff format app/ tests/

migrate:
    alembic upgrade head

run:
    uvicorn app.main:app --reload
```

Проблемы `make` в Python-проектах:

- **Табы vs пробелы:** `make` требует табуляции для команд. Пробелы — синтаксическая ошибка.
- **Shell-зависимость:** Makefile пишется под `sh` / `bash`. На Windows без WSL работает плохо.
- **Нет параметров:** сложно передать аргументы в команду (`make test path=tests/unit` — костыль).
- **Нет кроссплатформенности:** пути, переменные окружения,

команды отличаются.

Б.3.2. `just`: современная замена

`just` — командный раннер, написанный на Rust. Он решает все проблемы `make`:

- **Кроссплатформенность:** работает на Linux, macOS, Windows (PowerShell/Cmd).
- **Параметры:** удобные аргументы команд.
- **Явные зависимости:** рецепт может зависеть от другого.
- **Группировка:** рецепты можно группировать.
- **Переменные:** `.env` файл поддерживается из коробки.

```
In [ ]: # justfile
set dotenv-load := true

default:
    just --list

# Группа: разработка
[group('dev')]
install:
    uv sync

[group('dev')]
run:
    uv run uvicorn app.main:app --reload

# Группа: качество
[group('quality')]
lint:
    uv run ruff check app/ tests/
    uv run mypy app/

[group('quality')]
format:
    uv run ruff format app/ tests/
    uv run ruff check --fix app/ tests/

[group('quality')]
test *ARGS='':
    uv run pytest -xvs {{ARGS}}

# Группа: база данных
[group('db')]
migrate:
    uv run alembic upgrade head
```

```
[group('db')]
makemigrations name:
    uv run alembic revision --autogenerate -m "{{name}}"

# Группа: Docker
[group('docker')]
up:
    docker-compose up -d --build

[group('docker')]
down:
    docker-compose down -v
```

Использование:

```
In [ ]: just                # показать список команд
just test                # запустить тесты
just test tests/unit    # запустить только unit
just makemigrations "add users table"
just lint
```

Почему это важно: `just` — **исполняемая документация**. Новый разработчик клонирует репозиторий, видит `justfile`, и знает: `just install` -> `just migrate` -> `just run`. Нет необходимости читать README на 5 страниц.

Б.4. Контейнеризация: `docker`, `docker-compose`

(Детальное изложение Docker как технологии контейнеризации дано в Модуле 10. Здесь фокус на инструментарии разработчика.)

Б.4.1. Docker: среда выполнения как артефакт

Docker решает фундаментальную проблему: «работает на моей машине». Он упаковывает приложение со всеми зависимостями в **immutable image** — артефакт, который идентичен на ноутбуке разработчика, CI-сервере и продакшн-кластере.

Ключевые команды для разработчика:

```
In [ ]: # Сборка образа
docker build -t ml-api:latest .

# Запуск с пробросом портов и переменных окружения
```

```
docker run -p 8000:8000 -e DATABASE_URL=postgresql://... ml-api:latest

# Интерактивный вход для отладки
docker run -it --entrypoint /bin/bash ml-api:latest

# Удаление неиспользуемых образов
docker system prune -f
```

BuildKit: современный сборщик Docker (включён по умолчанию в Docker 23+). Поддерживает:

- **Параллельную сборку** слоёв.
- **Кэширование монтирований** (`--mount=type=cache,target=/root/.cache/pip`).
- **Secrets** (передача секретов на этап сборки без сохранения в слоях).

```
In [ ]: # Использование BuildKit cache для pip
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
```

Б.4.2. Docker Compose: оркестрация разработки

Docker Compose позволяет описать **многоконтейнерное** окружение в одном YAML-файле. Это стандарт для локальной разработки ML-сервисов.

```
In [ ]: # docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql+asyncpg://user:pass@db:5432/ml_db
      - REDIS_URL=redis://redis:6379
      - KAFKA_BROKER=kafka:9092
    volumes:
      - ./app:/app/app # hot-reload
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
```

```

    POSTGRES_DB: ml_db
volumes:
  - postgres_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U user -d ml_db"]
  interval: 5s
  timeout: 5s
  retries: 5

redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"

kafka:
  image: confluentinc/cp-kafka:latest
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
  depends_on:
    - zookeeper

zookeeper:
  image: confluentinc/cp-zookeeper:latest
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

worker:
  build:
    context: .
    dockerfile: Dockerfile.worker
  environment:
    - BROKER_URL=redis://redis:6379
  depends_on:
    - redis
    - db

volumes:
  postgres_data:

```

Команды:

```

In [ ]: docker compose up --build      # сборка и запуск всех сервисов
        docker compose up -d          # фоновый режим
        docker compose logs -f api    # следить за логами сервиса api
        docker compose exec db psql -U user -d ml_db # вход в БД
        docker compose down -v        # остановка + удаление volumes

```

Docker Compose Watch (новое в Docker Compose 2.24+):

автоматическая синхронизация файлов при изменении, без полного пересоздания контейнера. Быстрее, чем `volumes` для больших проектов.

```
In [ ]: services:
  api:
    build: .
    develop:
      watch:
        - action: sync
          path: ./app
          target: /app/app
        - action: rebuild
          path: ./pyproject.toml
```

Итог приложения Б

Инструмент	Задача	Когда использовать	Альтернатива
ruff	Линтинг + форматирование	Всегда. Заменяет flake8, isort, pylint	flake8 + isort + pylint
black	Строгое форматирование	Если нужна единая каноническая форма	ruff format (почти идентичен)
mypy	Статическая типизация	Всегда. Особенно с FastAPI/Pydantic	pyright, pyre
pre-commit	Клиентская валидация	Всегда. Fail-fast перед коммитом	husky (для JS), git hooks вручную
poetry	Управление зависимостями	Зрелые проекты, публикация в PyPI	pdm, uv
pdm	Управление зависимостями	Разработка библиотек, PEP-ориентированность	poetry, uv
uv	Управление зависимостями	Новые проекты, максимальная скорость	poetry, pip
just	Автоматизация команд	Всегда. Кроссплатформенность	make, invoke, npm scripts
make	Автоматизация команд	Если команда работает в Linux-only среде	just
docker	Контейнеризация	Всегда. Production + dev окружение	podman, buildah
docker compose	Многоконтейнерная разработка	Локальная разработка с БД, кэшем, брокерами	docker swarm, minikube

Золотой toolchain для нового ML-проекта на 2026 год:

```
In [ ]: # Инициализация
uv init ml-project
cd ml-project

# Зависимости
uv add fastapi pydantic sqlalchemy[asyncpg] redis httpx
uv add --dev pytest pytest-asyncio mypy ruff pre-commit
```

```
# Качество кода
ruff format .
ruff check --fix .
мypy app/

# Pre-commit
pre-commit install

# Автоматизация
# (создать justfile с командами test, lint, run, migrate)

# Контейнеризация
# (создать Dockerfile + docker-compose.yml по шаблону из Модуля 10)
```

Этот toolchain обеспечивает: детерминированные сборки, статическую безопасность типов, единый стиль кода, автоматическую валидацию на каждый коммит и идентичные окружения разработки и продакшна.